

Incremental Planarity Testing*

(Extended Abstract)

Giuseppe Di Battista**

Dipartimento di Informatica e Sistemistica
Università di Roma "La Sapienza"
Via Buonarroti, 12 – Rome, Italy 00185

Roberto Tamassia

Department of Computer Science
Brown University
Providence, Rhode Island 02912-1910

Summary

The *incremental planarity testing* problem consists of performing the following operations on a planar graph G with n vertices: (i) testing if a new edge can be added to G so that the resulting graph is itself planar; (ii) adding vertices and edges such that planarity is preserved. We present an efficient technique for incremental planarity testing that uses $O(n)$ space, and supports tests and insertions of vertices and edges in $O(\log n)$ time. The bounds for queries and vertex insertions are worst-case and the bound for edge insertions is amortized.

1 Introduction

The problems of testing planarity and constructing planar embeddings of graphs have been extensively studied in the past years and find direct application in a variety of areas including circuit layout, graphics, computer-aided design, and automatic graph drawing.

In a static environment, where an n -vertex graph G is entirely known in advance, we can test the planarity of G and compute a planar embedding in optimal $O(n)$ time [8]. In a dynamic environment, where a planar graph G is assembled on-line by insertions of vertices and edges, we would like to determine quickly whether an update causes G to become nonplanar. Namely, the *incremental planarity testing* problem consists of performing the following operations on a planar graph G : (i) testing if a new edge can be added to G so that the resulting graph is itself planar; (ii) adding vertices and edges such that planarity is preserved.

While many recent research efforts have been focused on planar graphs, the development of an efficient algorithm for incremental planarity testing has been an elusive goal. A first step in this direction has been the dynamic technique presented in [15] for the restricted problem of maintaining a planar embedding of a planar graph. This data structure uses $O(n)$ space, and allows to test whether two vertices are on the same face

of the embedding and to add vertices and edges to the embedding in $O(\log n)$ time. The related problem of computing a planar subgraph of a graph G with the maximum number of edges is NP-hard [5], and several heuristics have been developed for it, see, e.g., [3,11].

In this paper a technique for incremental planarity testing is presented that uses $O(n)$ space and supports tests and updates in $O(\log n)$ time. The following operations are defined on a 2-connected planar graph G :

Test(v_1, v_2): Determine whether edge (v_1, v_2) can be added to G while preserving planarity, i.e., test whether graph G admits an embedding Γ such that v_1 and v_2 are on the boundary of the same face of Γ .

InsertEdge(e, v_1, v_2): Add edge e between vertices v_1 and v_2 to graph G . The operation is allowed only if the resulting graph is itself planar.

InsertVertex(e, v, e_1, e_2): Split edge e into two edges e_1 and e_2 by inserting vertex v .

Embedding: Return a planar embedding Γ of G .

IncidenceList(v): Return the list of edges incident upon vertex v circularly sorted according to some embedding Γ of G .

The above repertory of operations is complete since any 2-connected planar graph G with m edges can be assembled starting from a graph consisting of a single edge by means of $m - 1$ *InsertEdge* and *InsertVertex* operations.

Our main result is expressed by the following theorem:

Theorem 1 *Let G be a planar 2-connected graph with n vertices. There exists a data structure for the incremental planarity testing problem in G with the following performance. The space requirement is $O(n)$. Operations *Test* and *InsertVertex* take worst-case time $O(\log n)$. Operation *InsertEdge* takes amortized time $O(\log n)$. Operation *Embedding* takes worst-case time $O(n)$. Operation *IncidenceList*(v) takes worst-case time $O(\log n + \deg(v))$.*

*Work partially supported by Cadre Technologies, Inc. (USA), and by Digital S.p.A. (Italy).

**Research performed in part while this author was visiting the Department of Computer Science of Brown University.

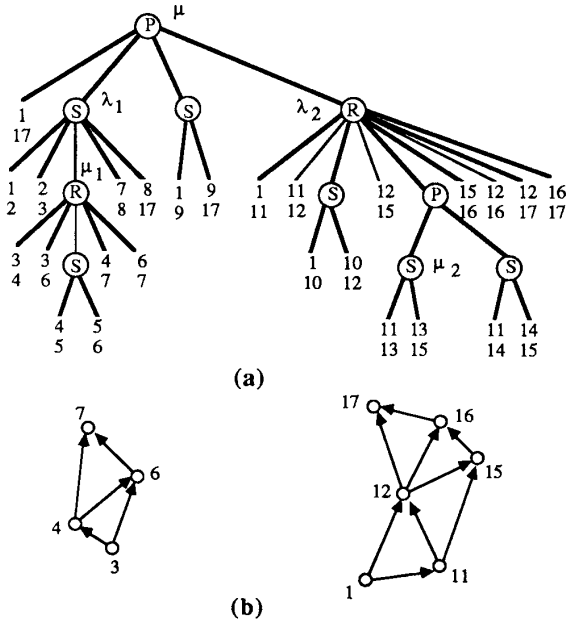


Figure 2: (a) Decomposition-tree T for the planar st -graph of Fig. 1 (the edges of the peripheral forest are drawn with bold lines). (b) Skeletons of the R-nodes.

$skeleton(\mu)$. Graph G_i is called the *pertinent graph* of node μ_i .

Fig. 2 illustrates the decomposition tree and the skeletons of the R-nodes for the planar st -graph of Fig. 1. Our definition of decomposition tree is a variation of the one given in [1] and is closely related to the decomposition of 2-connected graphs into 3-connected components presented in [7].

Lemma 1 *The decomposition tree T of G has $O(n)$ nodes. Also, the total number of edges of the skeletons stored at the nodes of T is $O(n)$.*

Let v be a vertex of G . The *allocation nodes* of v are the nodes of T whose skeleton contains v . Note that v has at least one allocation node.

Lemma 2 *Let v be a vertex of G distinct from s and t . The least-common-ancestor μ of the allocation nodes of v is itself an allocation node of v . Also, μ is the only allocation node of v such that v is not a pole of $skeleton(\mu)$.*

According to Lemma 2, the least-common-ancestor of the allocation nodes of vertex v is called the *proper allocation node* of v . Such a node is either an R-node or an S-node.

A *peripheral edge* (vertex) of a planar st -graph G is such that it appears on the external face of some embedding of G . A node μ of T is said to be peripheral if its virtual edge is peripheral in the skeleton of the parent of μ .

Let v_1 and v_2 be vertices of G , and μ_1 and μ_2 be the proper allocation nodes of v_1 and v_2 , respectively. If one of μ_1 and μ_2 is ancestor of the other, then we define μ as the lowest node on the path from μ_1 to μ_2 such that the pertinent graph of μ contains both v_1 and v_2 . Otherwise, we define μ as the least-common-ancestor of μ_1 and μ_2 .

If μ is an ancestor of μ_1 then let x_1 be the virtual edge in $skeleton(\mu)$ of the child λ_1 of μ on the path from μ to μ_1 , and let G_1 be the pertinent graph of λ_1 . Otherwise, let $x_1 = G_1 = v_1$. Define x_2 and G_2 similarly. Note that either one of x_1 and x_2 can be a vertex or an edge of $skeleton(\mu)$. The nodes μ_1 , μ_2 , μ , λ_1 , and λ_2 corresponding to $v_1 = 6$ and $v_2 = 13$ in Fig. 1 are shown in Fig. 2.

Theorem 2 *Let v_1 and v_2 be vertices of G . There exists an embedding Γ of G such that v_1 and v_2 are on the same face of Γ if and only if:*

1. x_1 and x_2 are on the same face of some embedding of $skeleton(\mu)$; and
2. v_1 and v_2 are peripheral vertices of G_1 and G_2 .

We remark that a skeleton graph either admits a unique embedding (R-node), or any two vertices/edges can be placed on the same face (P-, Q-, and S-nodes). Hence, Theorem 2 reduces the *Test* operation to a test on a fixed embedding. The following algorithm for operation *Test* is based on Theorem 2.

Algorithm $Test(v_1, v_2)$

1. Find the proper allocation nodes μ_1 of v_1 and μ_2 of v_2
2. Find the least-common-ancestor μ of μ_1 and μ_2 .
{ Due to space limitations we only describe the case when $\mu_1 \neq \mu$ and $\mu_2 \neq \mu$. }
3. **for** $i = 1, 2$ **do**
 determine the edge x_i of $skeleton(\mu)$ associated with μ_i , i.e., the virtual edge of the child of μ on the path from μ to μ_i
endfor
4. **for** $i = 1, 2$ **do**
 find the highest node ν_i of the path from μ_i to the root of T such that all the nodes of the path from μ_i to ν_i are peripheral
endfor

5. Return *true* iff all the following conditions are verified:

- (a) ν_1 is either a child of μ or is on the path from μ to the root of T ; and
- (b) ν_2 is either a child of μ or is on the path from μ to the root of T ; and
- (c) x_1 and x_2 are on the same face of some embedding of *skeleton*(μ). $\square$

The following data structures can be used to efficiently perform the *Test* operation. We store with each vertex v a pointer to its proper allocation node in T . Hence, Step 1 takes $O(1)$ time. We equip tree T with a data structure which uses linear space and supports least-common-ancestor queries in constant time [6,13]. Hence, Step 2 takes $O(1)$ time. Pointers from each node of T to the corresponding virtual edge allow us to perform Step 3 in $O(1)$ time. Additional pointers can be used to access nodes ν_1 and ν_2 of Step 4 in $O(1)$ time. Conditions (a) and (b) of Step 5 can be tested in $O(1)$ time by means of least-common-ancestor queries. Finally, we equip the skeleton of each R-node with the data structure for planar embedding tests described in [15]. This allows us to test Condition (c) of Step 5 in $O(\log n)$ time.

Theorem 3 *Let G be planar st-graph. There exists a data structure that supports operation $\text{Test}(u, v)$ on G in time $O(\log n)$, uses $O(n)$ space, and can be constructed in $O(n)$ preprocessing time.*

4 Updates

Due to space limitations, we present only the algorithm for operation *InsertEdge*(e, v_1, v_2). We assume that operation *Test*(v_1, v_2) has been already performed and that nodes μ_1, μ_2 , and their least common ancestor μ have been determined. We describe only the case when $\mu_1 \neq \mu$ and $\mu_2 \neq \mu$. The algorithm makes use of several types of transformations. If a transformation produces a node with one child, such node is absorbed into its parent.

Algorithm *InsertEdge*(e, v_1, v_2)

1. If μ is an S-node or a P-node and has more than two children, we perform one of the preliminary transformations at node μ illustrated in Fig. 3, where λ_1 and λ_2 are the children of μ on the paths from μ_1 to μ and from μ_2 to μ , respectively. The corresponding modifications of the skeletons are trivial.
2. **for** $i = 1, 2$ **do**
 set $\rho = \mu_i$;
 while $\rho \neq \mu$ **do**
 set π equal to the parent of ρ ;
 ElementaryTransformation ($\rho, \pi, v_i; \pi'$);
 set $\rho = \pi'$;

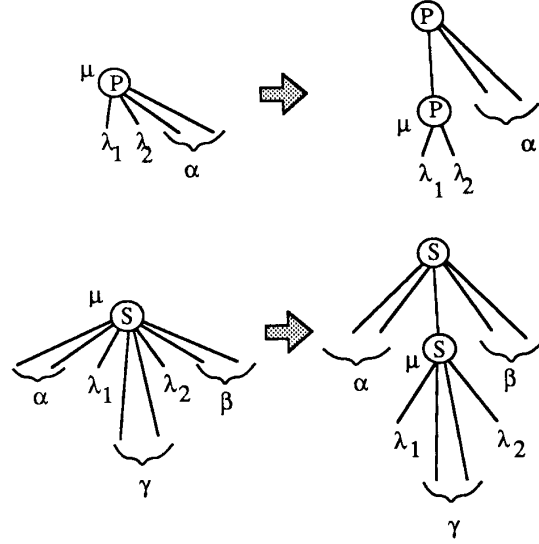


Figure 3: Preliminary transformations in Step 1 of Algorithm *InsertEdge*.

endwhile
endfor

3. Add edge e to *skeleton*(μ). $\square$

Procedure *ElementaryTransformation* ($\rho, \pi, v; \pi'$)

Let X and Y be the types of nodes ρ and π , respectively. Perform the XY -transformation shown in Fig. 4. Details are given below:

RR-transformation: Graph *skeleton*(π') is obtained from *skeleton*(π) by replacing the virtual edge of ρ with *skeleton*(ρ).

RP-transformation: Graph *skeleton*(ρ') is equal to *skeleton*(π) minus one edge. Graph *skeleton*(π') is equal to *skeleton*(ρ) plus a virtual edge between its poles.

RS-transformation: Let s_ρ and t_ρ be the poles of *skeleton*(ρ), and s_π and t_π be the poles of *skeleton*(π). Graph *skeleton*(π') consists of *skeleton*(ρ) plus edges from s_π to s_ρ and from t_ρ to t_π . Graphs *skeleton*(ρ') and *skeleton*(ρ'') are the subchains of *skeleton*(π) from s_π to s_ρ and from t_ρ to t_π , respectively.

SP-transformation: Let $\bar{s}$ and $\bar{t}$ be the common poles of *skeleton*(ρ) and *skeleton*(π). Graph *skeleton*(π') consists of the edges $(\bar{s}, v)$, $(v, \bar{t})$, and $(\bar{s}, \bar{t})$. Graph *skeleton*(ρ') is given by *skeleton*(π) minus one edge. Graphs *skeleton*(ρ'') and *skeleton*(ρ''') are the subchains of *skeleton*(ρ) from $\bar{s}$ to v and from v to $\bar{t}$, respectively.

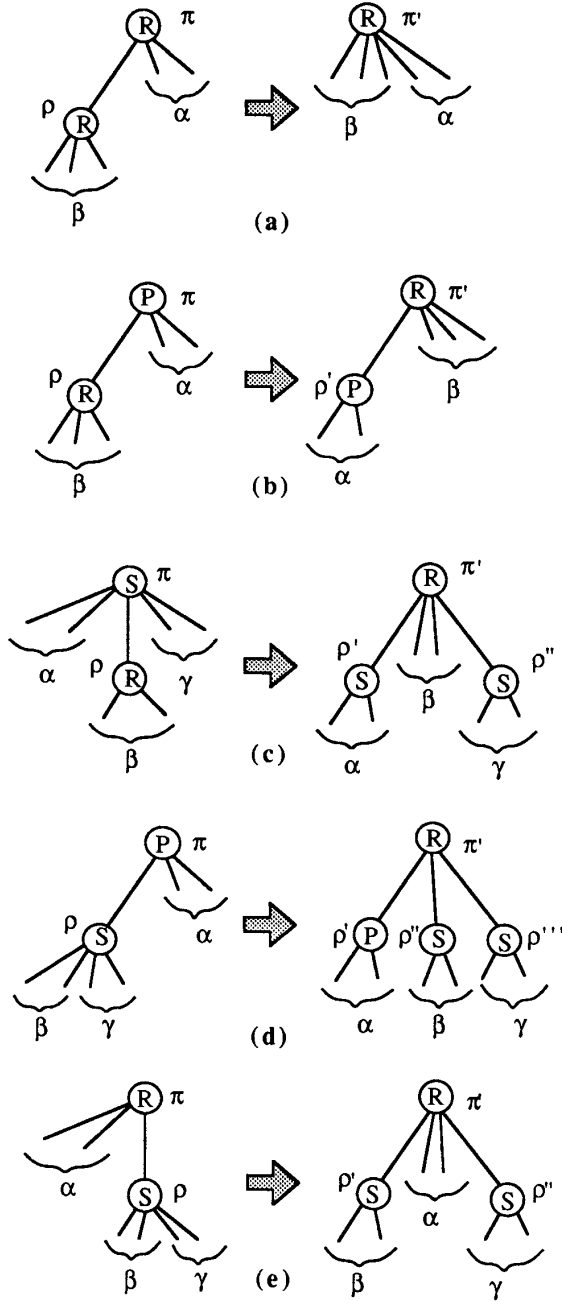


Figure 4: Elementary transformations: (a) RR; (b) RP; (c) RS; (d) SP; (e) SR.

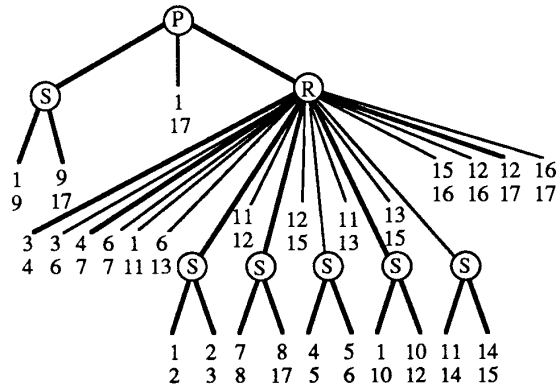


Figure 5: Tree T at the end of Algorithm *InsertEdge*, after the preliminary transformation at the root and the sequence of transformations: RS, RP, SP, RR, and RR.

SR-transformation: Let s_ρ and t_ρ be the poles of $\text{skeleton}(\rho)$, and let e_ρ (from s_ρ to t_ρ) be the virtual edge of ρ in $\text{skeleton}(\pi)$. Graph $\text{skeleton}(\pi')$ is obtained by replacing e_ρ with the chain (s_ρ, v, t_ρ) . Graphs $\text{skeleton}(\rho')$ and $\text{skeleton}(\rho'')$ are the subchains of $\text{skeleton}(\rho)$ from s_ρ to v and from v to t_ρ , respectively. $\square$

Fig. 5 shows tree T at the end of Algorithm *InsertEdge*, when the edge $(6,13)$ is inserted into the graph of Fig. 1.

Now, we describe the dynamic data structure. For each vertex v of G , we need to dynamically maintain information on the proper allocation node of v in T (Step 1 of *Test*). This is done by storing with each vertex v a pointer to its representative in the skeleton of its proper allocation node, and by storing with each node ν the vertices of $\text{skeleton}(\nu)$ in the leaves of a balanced search tree denoted by $\text{properiset}(\nu)$. For an S-node, the left-to-right order of the leaves of $\text{properiset}(\nu)$ corresponds to the bottom-to-top order of the vertices of $\text{skeleton}(\nu)$. For R-nodes and P-nodes, the order is arbitrary. Given a vertex of $\text{skeleton}(\nu)$ we determine ν by walking up to the root of the tree $\text{properiset}(\nu)$. Also, each elementary transformation on tree T corresponds to performing $O(1)$ split/splice operations on the trees $\text{properiset}(\nu)$. Hence, the dynamic maintenance of the sets $\text{properiset}(\nu)$ takes time $O(\log n)$.

To perform least-common-ancestor queries on T in a dynamic environment (Steps 2-3 of *Test*), we use a vari-

ation of the dynamic trees of Sleator and Tarjan [14] devised by Tarjan and Westbrook [18]. This data structure supports least-common-ancestor queries and generalized update operations (link/cut, expand/contract) on a collection of ordered trees in $O(\log n)$ time. Therefore, the update of tree T after each transformation takes $O(\log n)$ time.

Regarding Step 4 of the *Test* algorithm, for each node μ of T we need to dynamically maintain information about the highest node ν on the path from μ to the root of T such that all the nodes of the path from μ to ν are peripheral. For this purpose, we define the *peripheral forest* $\mathcal{F}$ of T , which is obtained by removing from T all arcs (ρ, π) such that π is the parent of ρ and is not a peripheral node. Node ν associated with μ is the root of the tree of $\mathcal{F}$ that contains node μ . Edges of the peripheral forests in the decomposition trees are drawn with bold lines in Figs. 2 and 5.

The forest $\mathcal{F}$ is updated in the various elementary transformations by means of link/cut and expand/contract operations. Hence, by representing $\mathcal{F}$ by the aforementioned variation of dynamic trees, the update of forest $\mathcal{F}$ after each transformation takes $O(\log n)$ time.

Regarding Step 5 of the *Test* algorithm, checking Condition (c) requires that we represent the skeleton of each node with a data structure for dynamic planar embedding. For this purpose, we use a variation of the data structure of [15], which supports the operation of replacing an edge with a subgraph (RR-transformation). The update of this data structure in each elementary transformation can be done in time $O(\log n)$.

Theorem 4 *The above data structure for incremental planarity testing uses $O(n)$ space.*

Theorem 5 *Each elementary transformation of the decomposition tree T in the *InsertEdge* algorithm can be performed in $O(\log n)$ time.*

Theorem 6 *The amortized time complexity of operations *InsertEdge* and *InsertVertex* over a sequence of such operations is $O(\log n)$.*

Sketch of Proof: Let N be the total number of internal (R-, S-, and P-) nodes of T , D_R be the total number of children of all R-nodes, and D_{SP} be the total number of children of all S-nodes and P-nodes. We define the following *potential function* associated with the data structure:

$$\Phi = 2N + D_{SP} - D_R.$$

Let T be the number of elementary transformations performed in an *InsertEdge* or *InsertVertex* operation, and define the amortized number of elementary transformations as $A = T + \Delta\Phi$. We show that $A = O(1)$ for both *InsertEdge* and *InsertVertex* operations. Since $|\Phi| = O(n)$, the total complexity of a sequence of n *InsertEdge* and *InsertVertex* operations starting from a graph with $O(1)$ vertices is $O(n \log n)$. $\square$

5 References

- [1] D. Bienstock & C.L. Monma, "On the Complexity of Covering Vertices by Faces in a Planar Graph," *SIAM J. Computing* 17 (1988), 53–76.
- [2] G. Di Battista & R. Tamassia, "Algorithms for Plane Representations of Acyclic Digraphs," *Theoretical Computer Science* 61 (1988), 175–198.
- [3] P. Eades, L. Foulds & J. Giffin, "An Efficient Heuristic for Identifying a Maximal Weight Planar Subgraph," *Combinatorial Mathematics IX, Lecture Notes in Mathematics* 952 (1982), 239–251.
- [4] S. Even & R.E. Tarjan, "Computing an st-Numbering," *Theoretical Computer Science* 2 (1976), 339–344.
- [5] M.R. Garey & D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, 1979.
- [6] D. Harel & R.E. Tarjan, "Fast Algorithms for Finding Nearest Common Ancestors," *SIAM J. Computing* 13 (1984), 338–355.
- [7] J. Hopcroft & R.E. Tarjan, "Dividing a Graph into Triconnected Components," *SIAM J. Computing* 2 (1973), 135–158.
- [8] J. Hopcroft & R.E. Tarjan, "Efficient Planarity Testing," *J. ACM* 21 (1974), 549–568.
- [9] A. Lempel, S. Even & I. Cederbaum, "An Algorithm for Planarity Testing of Graphs," *Theory of Graphs, Int. Symposium* (1966), 215–232.
- [10] T. Nishizeki & N. Chiba, *Planar Graphs: Theory and Algorithms*, Annals of Discrete Mathematics 32, North-Holland, 1988.
- [11] T. Ozawa & H. Takahashi, "A Graph-planarization Algorithm and its Applications to Random Graphs," *Graph Theory and Algorithms, Lecture Notes in Computer Science* 108 (1981), 95–107.
- [12] F.P. Preparata & R. Tamassia, "Fully Dynamic Techniques for Point Location and Transitive Closure in Planar Structures," *Proc. 29th IEEE Symp. on Foundations of Computer Science* (1988), 558–567.
- [13] B. Schieber & U. Vishkin, "On Finding Lowest Common Ancestors: Simplification and Parallelization," *SIAM J. Computing* 17 (1988), 1253–1262.
- [14] D.D. Sleator & R.E. Tarjan, "A Data Structure for Dynamic Trees," *J. Computer Systems Sciences* 24 (1983), 362–381.
- [15] R. Tamassia, "A Dynamic Data Structure for Planar Graph Embedding," *Automata, Languages and Programming (Proc. 15th ICALP), Lecture Notes in Computer Science* 317 (1988), 576–590.
- [16] R. Tamassia & F.P. Preparata, "Dynamic Maintenance of Planar Digraphs, with Applications," *Algorithmica* (1989, to appear).
- [17] R. Tamassia & I.G. Tollis, "A Unified Approach to Visibility Representations of Planar Graphs," *Discrete & Computational Geometry* 1 (1986), 321–341.
- [18] R.E. Tarjan, "Personal Communication," 1989.